Reporte Técnico de Actividades Práctico Experimentales Nro 005

1. Datos de identificación del estudiante y la práctica

Campo	Información
Tipo	Informe individual
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	05
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Nombre del Estudiante	Yandri A. Piscocama J.
Fecha	29 y 30 de octubre de 2026
Horario	Jueves 10h30 a 11h30; viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivos de la práctica

Objetivo general. Comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.

• Implementar un generador LCG en Python.

• Implementar el mismo algoritmo en Java.

• Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales y reactivos:

Computador.
Python.
Java/JDK.

4. Equipos y herramientas

Python
JDK.
Sistema operativo Linux o macOS

5. Procedimiento y metodología ejecutada

5.1. Fundamento del generador

El generador congruencial lineal (LCG) construye una secuencia determinista a partir de una semilla, un multiplicador, un incremento y un módulo. Se utilizó la recurrencia $X(n+1) = (a \cdot X(n)+c) \bmod m$ y la normalización $U(n+1) = X(n+1)/m$, que produce valores en [0,1). La elección de parámetros determina el período y otras propiedades de la secuencia [1], [2].

5.2. Parámetros y procedimiento

Parámetro	Valor
Semilla principal	12345

Multiplicador a	1664525
Incremento c	1013904223
Módulo m	2^{32} = 4294967296
Cantidad N	10000
Semilla alternativa	42

1. Se revisó la guía y se fijaron los parámetros del LCG para ambas implementaciones.

2. Se implementó la misma recurrencia en Python y Java; los resultados se normalizaron y exportaron a archivos CSV.

3. Se generaron 10 000 valores con la semilla 12345 y se compararon las primeras 20 observaciones de ambos lenguajes.

4. Se comprobó la coincidencia de las secuencias completas, se elaboró la gráfica con Matplotlib y se analizó el período a partir de las condiciones de Hull–Dobell.

5. Se repitió la generación cambiando únicamente la semilla a 42 para estudiar cómo cambia la secuencia.

6. Resultados obtenidos

6.1. Comparación Python–Java

La tabla siguiente presenta las primeras 20 observaciones normalizadas obtenidas con la semilla 12345. Los estados enteros y los valores normalizados de ambos programas coincidieron; los archivos CSV completos también fueron iguales.

Tabla 1. Primeras veinte observaciones generadas en Python y Java.

N	Python	Java	Coinciden
1	0,02040269	0,02040269	Sí
2	0,01654785	0,01654785	Sí
3	0,54315579	0,54315579	Sí
4	0,63490406	0,63490406	Sí
5	0,91002951	0,91002951	Sí
6	0,11246167	0,11246167	Sí
7	0,49588941	0,49588941	Sí
8	0,54834884	0,54834884	Sí
9	0,59610012	0,59610012	Sí
10	0,78345042	0,78345042	Sí
11	0,03985432	0,03985432	Sí
12	0,74380122	0,74380122	Sí
13	0,96580937	0,96580937	Sí
14	0,07366754	0,07366754	Sí
15	0,69960693	0,69960693	Sí
16	0,46029417	0,46029417	Sí
17	0,39648352	0,39648352	Sí
18	0,95952129	0,95952129	Sí
19	0,40612530	0,40612530	Sí
20	0,95276416	0,95276416	Sí

Las veinte observaciones de la tabla coinciden y la comparación de los CSV confirma la equivalencia de los 10 000 registros generados para cada semilla. Este resultado valida la reproducibilidad entre las implementaciones, pero no demuestra por sí solo la calidad estadística de la secuencia.

6.2. Visualización de la secuencia

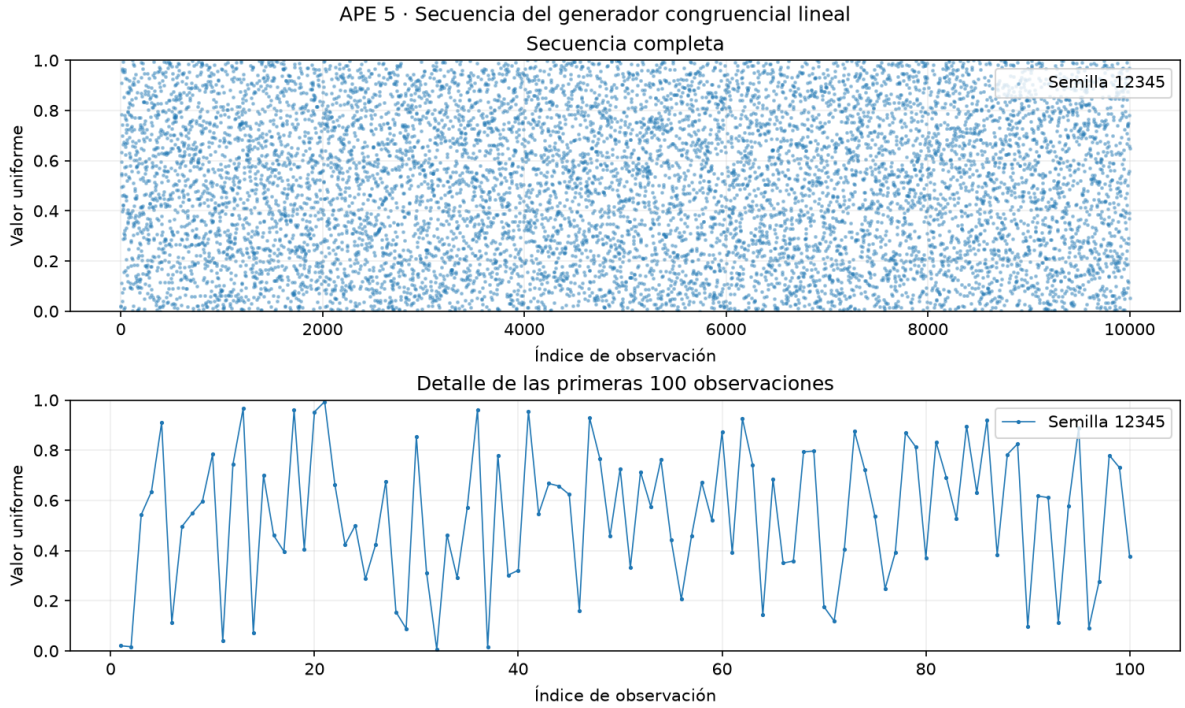


Figura 1. Secuencia LCG: 10 000 valores y detalle de las primeras 100 observaciones.

La figura permite observar variaciones de los valores normalizados dentro del intervalo [0,1). El detalle de las primeras 100 observaciones muestra cambios entre valores consecutivos. La apariencia dispersa de la gráfica no basta para afirmar independencia ni verdadera aleatoriedad [2].

6.3. Análisis del período

Para el LCG mixto se verificaron las condiciones de Hull–Dobell: el incremento 1013904223 es coprimo con el módulo 2^{32}; el único factor primo del módulo es 2 y divide a−1 = 1664524; además, como el módulo es divisible por 4, también lo es a−1. Por tanto, el período es completo: 2^{32} = 4 294 967 296 estados [1]. No fue necesario recorrer todos los estados para obtener esta conclusión.

6.4. Comparación del cambio de semilla

Tabla 2. Primeros diez valores con las semillas 12345 y 42.

N	Semilla 12345	Semilla 42
1	0,02040269	0,25234517
2	0,01654785	0,08812505
3	0,54315579	0,57728120
4	0,63490406	0,22255427
5	0,91002951	0,37566020
6	0,11246167	0,02566390
7	0,49588941	0,44728129
8	0,54834884	0,11846000
9	0,59610012	0,87381371
10	0,78345042	0,99463428

La semilla 12345 produjo como primer estado 87628868 ($U_1 \approx 0{,}02040269$), mientras que la semilla 42 produjo 1083814273 ($U_1 \approx 0{,}25234517$). Así, cambiar la semilla altera el punto de partida y los valores observados, pero conserva el período completo porque los demás parámetros siguen siendo los mismos.

7. Preguntas de control

1. ¿Qué significa pseudoaleatorio?

Se refiere a una secuencia producida por un algoritmo determinista que imita ciertas propiedades de la aleatoriedad. Con la misma semilla y parámetros se obtiene la misma secuencia [1].

2. ¿Qué función cumple la semilla?

Define el estado inicial del generador. Su cambio modifica la secuencia obtenida, mientras que mantenerla permite repetir el experimento [1].

3. ¿Qué significa período?

Es la cantidad de estados que recorre un generador antes de repetir uno y, desde allí, volver a producir el mismo ciclo [1].

4. ¿Qué ocurre si se utilizan parámetros inadecuados?

Pueden aparecer ciclos cortos, valores repetidos demasiado pronto o patrones indeseables. Un período largo, por sí solo, tampoco asegura independencia [1], [2].

5. ¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?

Permite comprobar la reproducibilidad y detectar posibles diferencias de aritmética, normalización o exportación. Aquí las secuencias de Python y Java coincidieron.

8. Conclusiones

1. La implementación del LCG en Python y Java permitió generar 10 000 valores pseudoaleatorios con resultados coincidentes al emplear los mismos parámetros. La comparación de las primeras veinte observaciones y de los archivos completos confirmó la reproducibilidad de ambos programas.

2. El análisis de las secuencias permitió comprobar que el generador presenta un período completo de 2^{32} estados con los parámetros seleccionados. Al cambiar la semilla se modificaron los valores iniciales sin alterar ese período, lo que permitió comprender la influencia de la semilla y la importancia de elegir adecuadamente los parámetros.

9. Referencias bibliográficas

[1] D. E. Knuth, The Art of Computer Programming, vol. 2, Seminumerical Algorithms, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[2] P. L'Ecuyer, "Tables of linear congruential generators of different sizes and good lattice structure," Mathematics of Computation, vol. 68, no. 225, pp. 249–260, 1999, doi: 10.1090/S0025-5718-99-00996-5.

10. Anexos

10.1. Repositorio del proyecto

Simulación Portafolio en GitHub: https://github.com/YandriPiscocama/Simulacion_Portafolio

10.2. Declaración de uso de inteligencia artificial

Se utilizó Codex (OpenAI) como apoyo para implementar el generador congruencial lineal en Python y Java y preparar las pruebas automatizadas. El estudiante especificó los parámetros y los objetivos de la práctica y revisó las secuencias, las tablas y la gráfica obtenidas. También se utilizó ChatGPT como apoyo para organizar el informe, interpretar resultados y revisar la redacción y las referencias.

10.3. Resúmenes de prompts utilizados

Los siguientes textos resumen las instrucciones empleadas y no constituyen transcripciones literales de las conversaciones.

1. Generación del código en Python y Java. Implementa un generador LCG configurable con los parámetros de la guía, genera 10 000 valores en ambos lenguajes y exporta las secuencias a CSV. Valida la coincidencia de los primeros 20 registros y de las muestras completas.

2. Visualización y análisis de resultados. Grafica la secuencia LCG con Matplotlib, justifica su período mediante las condiciones de Hull–Dobell y compara el efecto de utilizar las semillas 12345 y 42, explicando los resultados observados.